

AFRILANG Stdlib

std/c/netcidr

Français. Bibliothèque AFRILANG 'std/c/netcidr' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : prefixLen, hostCount, maskFromPrefix, ipToInt, networkInt, containsIp, broadcastInt.

```
'import "std/c/netcidr"' · 'use netcidr'
```

```
- 'prefixLen(cidr text) → number' - 'hostCount(cidr text) → number' -  
'maskFromPrefix(prefix number) → number' - 'ipToInt(ip text) → num-  
ber' - 'networkInt(cidr text) → number' - 'containsIp(cidr text, ip text) →  
bool' - 'broadcastInt(cidr text) → number'
```

English. AFRILANG library 'std/c/netcidr' (tier complex, category complex). Exports 7 function(s): prefixLen, hostCount, maskFromPrefix, ipToInt, networkInt, containsIp, broadcastInt.

```
'import "std/c/netcidr"' · 'use netcidr'
```

```
- 'prefixLen(cidr text) → number' - 'hostCount(cidr text) → number' -  
'maskFromPrefix(prefix number) → number' - 'ipToInt(ip text) → num-  
ber' - 'networkInt(cidr text) → number' - 'containsIp(cidr text, ip text)  
→ bool' - 'broadcastInt(cidr text) → number'
```

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	7	

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/netcidr' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : prefixLen, hostCount, maskFromPrefix, ipToInt, networkInt, containsIp, broadcastInt.

```
'import "std/c/netcidr"' · 'use netcidr'
```

```
- `prefixLen(cidr text) → number`
- `hostCount(cidr text) → number`
- `maskFromPrefix(prefix number) → number`
- `ipToInt(ip text) → number`
- `networkInt(cidr text) → number`
- `containsIp(cidr text, ip text) → bool`
- `broadcastInt(cidr text) → number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/netcidr"
use netcidr
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- prefixLen(cidr text) → number•
hostCount(cidr text) → number•
maskFromPrefix(prefix number) → number•
ipToInt(ip text) → number•
networkInt(cidr text) → number•
containsIp(cidr text, ip text) → bool•
broadcastInt(cidr text) → number

4. Exemples d'utilisation

```
import "std/c/netcidr"
use netcidr
```

```
create result = prefixLen(/* args */)
say result
```

```
create result = hostCount(/* args */)
say result
```

```
create result = maskFromPrefix(/* args */)
say result
```

```
create result = ipToInt(/* args */)
say result
```

```
create result = networkInt(/* args */)
say result
```

```
create result = containsIp(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/netcidr"`` en tête de fichier.
- Lancez ``afriLang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module netcidr

```
function prefixLen(cidr text) returns number
end
```

```
function hostCount(cidr text) returns number
end
```

```
function maskFromPrefix(prefix number) returns number
end
```

```
function ipToInt(ip text) returns number
end
```

```
function networkInt(cidr text) returns number
end
```

```
function containsIp(cidr text, ip text) returns bool
end
```

```
function broadcastInt(cidr text) returns number
end
end
```

English

1. Overview

AFRILANG library 'std/c/netcidr' (tier complex, category complex). Exports 7 function(s): prefixLen, hostCount, maskFromPrefix, ipToInt, networkInt, containsIp, broadcastInt.

```
'import "std/c/netcidr"' · 'use netcidr'
```

```
- `prefixLen(cidr text) → number`  
- `hostCount(cidr text) → number`  
- `maskFromPrefix(prefix number) → number`  
- `ipToInt(ip text) → number`  
- `networkInt(cidr text) → number`  
- `containsIp(cidr text, ip text) → bool`  
- `broadcastInt(cidr text) → number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/netcidr"  
use netcidr
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- prefixLen(cidr text) → number•
hostCount(cidr text) → number•
maskFromPrefix(prefix number) → number•
ipToInt(ip text) → number•
networkInt(cidr text) → number•
containsIp(cidr text, ip text) → bool•
broadcastInt(cidr text) → number

4. Usage examples

```
import "std/c/netcidr"  
use netcidr
```

```
create result = prefixLen(/* args */)   
say result
```

```
create result = hostCount(/* args */)   
say result
```

```
create result = maskFromPrefix(/* args */)   
say result
```

```
create result = ipToInt(/* args */)   
say result
```

```
create result = networkInt(/* args */)
say result
```

```
create result = containsIp(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/netcidr"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module netcidr

```
function prefixLen(cidr text) returns number
end
```

```
function hostCount(cidr text) returns number
end
```

```
function maskFromPrefix(prefix number) returns number
end
```

```
function ipToInt(ip text) returns number
end
```

```
function networkInt(cidr text) returns number
end
```

```
function containsIp(cidr text, ip text) returns bool
end
```

```
function broadcastInt(cidr text) returns number
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/netcidr"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/netcidr/>

Source (excerpt)

```
module netcidr

function prefixLen(cidr text) returns number
end

function hostCount(cidr text) returns number
end

function maskFromPrefix(prefix number) returns number
end

function ipToInt(ip text) returns number
end

function networkInt(cidr text) returns number
end

function containsIp(cidr text, ip text) returns bool
end

function broadcastInt(cidr text) returns number
end
end
```